



TM642

Diagnostics, charts and customized widgets

Requirements

Training modules	TM611 - Working with mapp View TM671 - Creating efficient mapp View HMI applications
Software	Requirement for this training module is the mapp View project created in TM611. Automation Studio 6.0.2 Automation Runtime 6.0 mapp View 6.0.0
Hardware	ARsim

Table of contents

1 Introduction.....	4
1.1 Learning objectives.....	4
1.2 Symbols and safety notices.....	4
2 mapp View diagnostics.....	5
2.1 Automation Studio Logger window.....	5
2.2 mapp View diagnostics page.....	7
2.3 mapp View performance analysis.....	11
3 Displaying diagrams.....	16
3.1 OnlineChartHDA.....	16
3.2 XYChart.....	22
4 Customized widgets.....	28
4.1 Widget library.....	28
4.2 Derived widgets.....	29
4.3 Compound widget.....	30
4.4 Custom keyboards.....	39
5 Summary.....	40

1 Introduction

Intuitive operation

Modern web technology enables completely new operating concepts for machines and systems. Efficient and intuitive operation increases productivity and reduces downtimes.



The tasks in this training module build upon existing basic knowledge of mapp View. In particular, the module covers integrated diagnostic access and its use, as well as the display of process values in trend curves. Additionally, widgets are used that are individually adapted to customer requirements.

1.1 Learning objectives

This training module covers advanced topics that represent typical use cases of a mapp View application. Numerous exercises that demonstrate the configuration and usage increase the efficiency of the learning process.

- Participants will receive an overview of the integrated diagnostic tools.
- Participants will be able to test a mapp View HMI application using the integrated diagnostics.
- Participants will be able to identify an error situation using the mapp View diagnostics page.
- Participants will become familiar with the guidelines and recommendations for optimal performance of an HMI application.
- Participants will be able to analyze the performance of the HMI application.
- Participants will be able to display a live signal in widget "OnlineChartHDA".
- Participants will be able to display x-values and y-values using two data arrays in widget "XYChart".
- Participants will be able to stop and analyze the data of the two chart widgets.
- Participants will be able to create their own derived and composite widgets in a widget library.

1.2 Symbols and safety notices

Unless otherwise specified, the symbol descriptions and safety notices listed in "TM210 - Working with Automation Studio" apply.

2 mapp View diagnostics

This section contains information about troubleshooting a mapp View HMI application.

The following tools are available for diagnostics of a mapp View HMI application:

- [2.1 "Automation Studio Logger window" on page 5](#)
- [2.2 "mapp View diagnostics page" on page 7](#)
- [2.3 "mapp View performance analysis" on page 11](#)

2.1 Automation Studio Logger window

The Logger enables the display of events that occur within an application on a controller. mapp View events are entered in the "Visualization" module.

In an earlier exercise with an event binding, it was observed that write access to an OPC UA variable without write permissions had no effect in the HMI application. Such situations occur especially when the operator of an HMI application can operate elements – regardless of whether they have the appropriate rights to do so. This situation can be analyzed in the Logger.



Figure 1: Modifying an OPC UA value without write access.

Exercise: Detect an error situation in Automation Studio Logger

The goal of this task is to detect an error situation using the Logger in Automation Studio. The OPC UA variable "Set-Temperature" is not set to the value defined in the event binding (35).

- 1) Log out the user.
- 2) Set variable "SetTemperature" to a value >35 in the Watch window.
- 3) Press the "ButtonSetToDefault" button.
- 4) Analyze the error in the Logger.



To display the mapp View error messages, the "Visualization" filter must be enabled and the entries updated.



The "Visualization" filter is enabled in the Logger and new entries are loaded continuously. The "Details" window clearly displays the relevant information, such as error messages.

The screenshot shows the 'Logger' application interface. On the left, the 'Modules' tree is expanded, and the 'Visualization' module is selected. The main window displays a list of log entries with columns for Severity, Time, ID, Entered by, and Description. The entry with ID -1061091064 is highlighted, showing an error message: 'Failed to unsubscribe to OPC UA node attribute (id: NS5String): AeGlobalPV.SetTemperature. attribute: Value, serverAlias:). Status: BadUserAccessDenied'.

Severity	Time	ID	Entered by	Description
1 Error	2026-01-29 10:29:41.155600	-1061091064	mapp View Server	Problem executing action "opcUaNodeAction.SetValueNumber" in context "ContentMain.Button"
2 Error	2026-01-29 10:29:41.155600	-1061091064	mapp View Server	Failed to unsubscribe to OPC UA node attribute (id: NS5String): AeGlobalPV.SetTemperature. attribute: Value, serverAlias:). Status: BadUserAccessDenied
3 Success	2026-01-29 10:29:14.930400	12650797	mapp View Server	Established connection to configured OPC UA server (serverAlias: "").
4 Success	2026-01-29 10:29:14.714800	12680448	mapp View Client	Client initialization successful. Client ID: "b4b27bb73edc378bae035e3a9f92a51"
5 Success	2026-01-29 10:29:14.611200	12650458	mapp View Server	Client with ID specified in text was connected.
6 Information	2026-01-29 10:29:10.670400	1086390773	mapp View Server	mapp View server has been started.
7 Information	2026-01-29 10:29:10.245600	1086391292	mapp View Server	mapp View version used: 6.5.0
8 Information	2026-01-29 10:29:10.235200	1086390772	mapp View Server	mapp View server is being started.
9 Success	2026-01-29 10:29:09.964000	3157279	ROOT	Project installation completed successfully
10 Information	2026-01-29 10:29:09.964000	1076899103	ROOT	Installation settings
11 Information	2026-01-29 10:29:09.702000	31280	ROOT	AR logger module created
12 Warning	2026-01-29 10:29:04.448000	30028	ROOT	Carried out reboot
13 Information	2026-01-29 10:29:04.448000	31280	ROOT	AR logger module created

The 'Details' window shows the following information for the selected error entry:

Name	Value
Severity	Error
Date	29.01.2026
Time	10:29:41.155600
ID	-1061091064
Area	B&R
Facility	193
Entered by	mapp View Server
Logbook	Visualization
Record ID	8
Format	EventId
Location	Online
Description	Failed to unsubscribe to OPC UA node attribute (id: NS5String): AeGlobalPV.SetTemperature. attribute: Value, serverAlias:). Status: BadUserAccessDenied
ASCII Data	21 4E 53 36 7C 53 74 72 69 6E 67 7C 3A 3A 41 73 47 65 6F 62 61 6C 50 56 3A 53 65 74 54 65 6D 70 65 72 61 74 75 72 65 00 21 56 61 6C 75 65 00 21 00 21 42
Binary Data	61 64 55 73 65 72 41 63 63 65 73 73 44 65 6E 69 65 64 00 FF

Figure 2: OPC UA error message when writing to variable "SetTemperature".

2.2 mapp View diagnostics page

mapp View enables troubleshooting of the HMI application in a browser. This requires a connection to the mapp View server and the correct URL: **IP address:81/server/info**.

The following information is displayed on the diagnostics pages:

- General information about all clients connected to the mapp View server
- Values for active bindings for a specific client
- Values of session variables
- Expressions of an active piece of content
- Chronological display of events from different sources and their executed actions



Access to the diagnostics page from the browser is only possible for configured roles, i.e. a login window for entering the username and a password is automatically displayed when accessing the page.



[HMI \ mapp View \ Guides \ Diagnostics \ Diagnostics page](#)

[HMI \ mapp View \ Engineering \ Organization of the HMI application \ mapp View configuration](#)

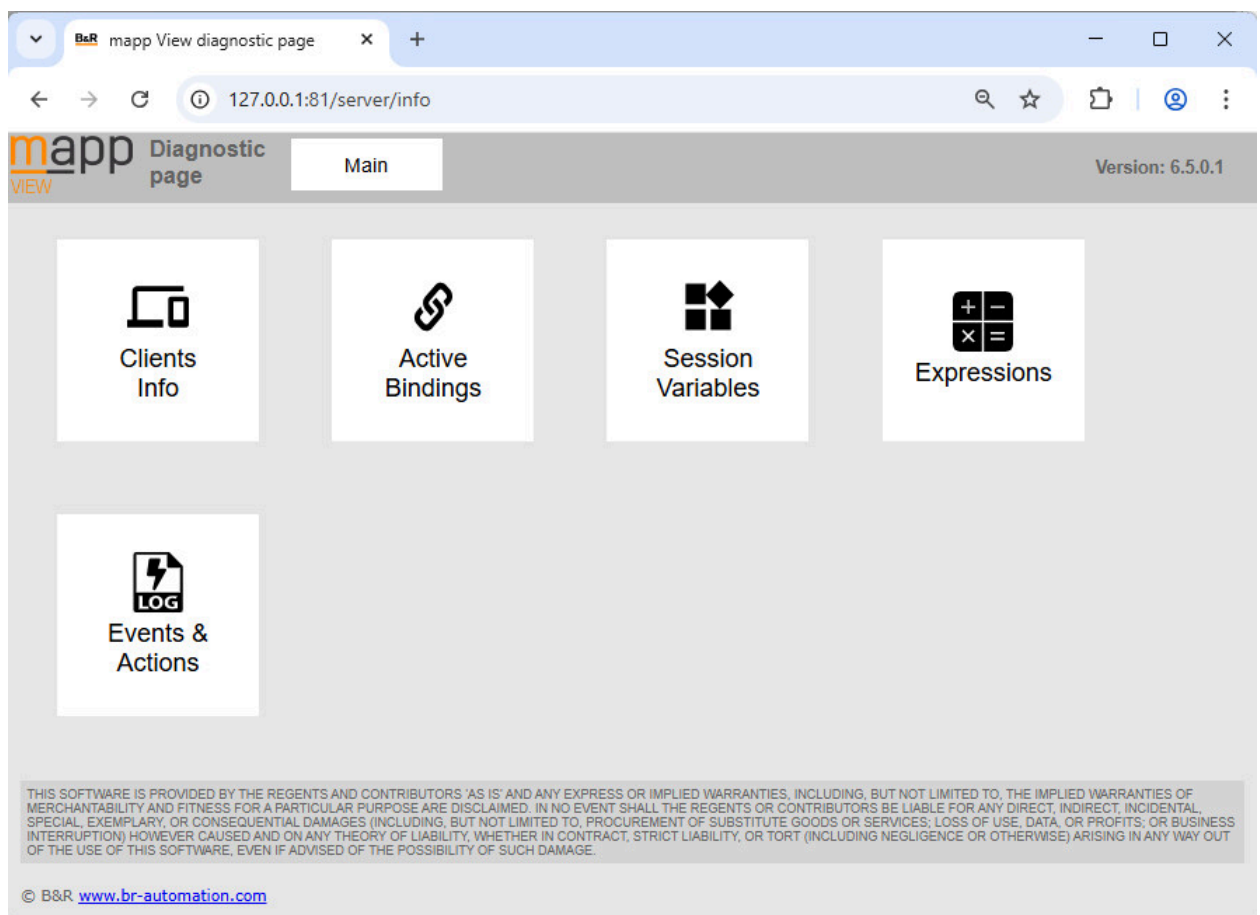


Figure 3: Diagnostics page in browser after successful login

Exercise: Enable diagnostics page

The diagnostics page of the mapp View server is disabled by default. To access the diagnostics page from the browser, it must be enabled in the mapp View configuration.

- 1) Open the mapp View configuration.
- 2) Enable the diagnostics page.
- 3) Assign the "Service" role.



The mapp View configuration is configured so that any user with the "Service" role can open the diagnostics page.

Name	Value	Description
MapViewConfiguration		
Server configuration		
Protocol	HTTP	Use secure communication protocol.
Maximal client connections	3	Number of maximum permitted client connections in
Use user preferences from MpUserX	FALSE	Use user preferences such as language and meas
Startup User	fixed user	Defines logged in user at startup.
Diagnostic Configuration		
Activate Diagnostic Page	on	Enable access to diagnostics page
Role 1	Service	Role permitting User to access diagnostic page
Role 2		Role permitting User to access diagnostic page
OPC UA system		

Figure 4: Advanced parameters view in the mapp View configuration

Exercise: Open the diagnostics page in a browser

- 1) Compile the project and transfer it to ARsim.
- 2) Open the browser (Google Chrome).
- 3) Enter URL: <http://127.0.0.1:81/server/info>
- 4) Log in user with Service role.



When the diagnostic page is opened in the browser, the user with the configured role must log in with their credentials in order to view the content.

mapp View diagnostic page x +

localhost:81/server/info

Sign in
 http://127.0.0.1:81
 Username: UserService
 Password:
 Sign in Cancel

Figure 5: Login dialog box for access to the diagnostics page.

Exercise: Identify an error situation on the diagnostics page

The goal of this task is to detect an error situation using the mapp View diagnostics page in the browser. The OPC UA variable "SetTemperature" is not set to the value defined in the event binding (35).

- 1) Log out the user.
- 2) Set variable "SetTemperature" to a value >35 in the Watch window.
- 3) Navigate to the page "Events & Actions".
- 4) Start the recording.

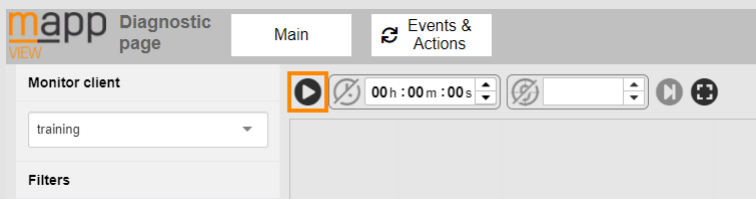


Figure 6: Start recording events and actions.

- 5) Press the "ButtonSetToDefault" button.
- 6) Analyze errors on the diagnostic page "Events & Actions".



After the recording has ended, the events that have occurred and the actions configured in the event binding are displayed chronologically.

(2)		Time	Scope	RefId	Name
1	+	2026-01-29 10:33:47.519	training - ContentMain	ButtonResetTemp	Click
2	+	2026-01-29 10:33:47.519	training - ContentMain	::AsGlobalPV:SetTemperature	SetValueNumber

THIS SOFTWARE IS PROVIDED BY THE REGENTS AND CONTRIBUTORS 'AS IS' AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES, LOSS OF USE, DATA, OR PROFITS, OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

© B&R www.br-automation.com

Figure 7: Chronological display of events and actions.

Exercise: Test various events

The "Events & Actions" diagnostics page can be used to test further event bindings configured in the HMI application.

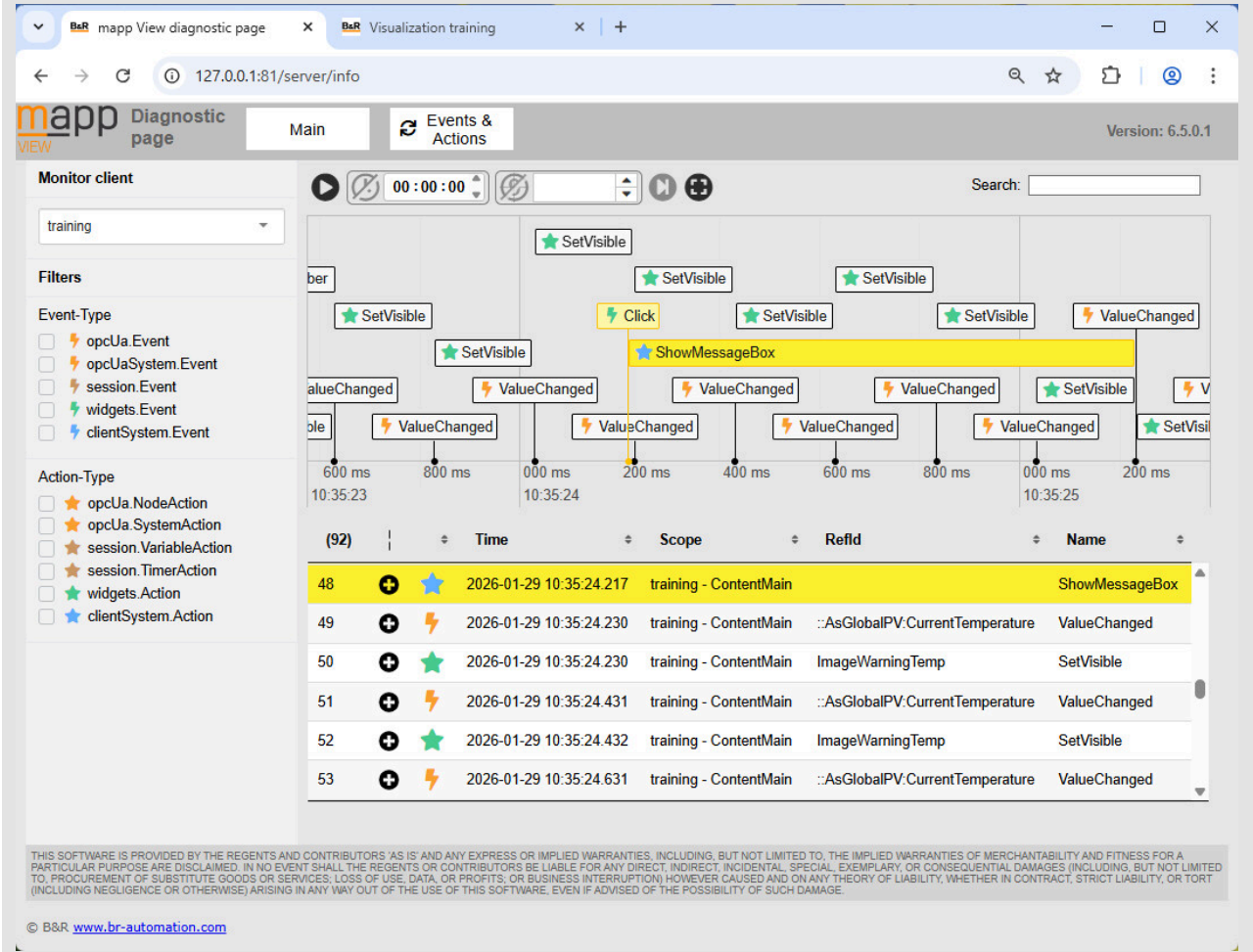


Figure 8: Analysis of various events and their actions.

2.3 mapp View performance analysis

Automation Help contains guides in the mapp View section. These guides are very helpful when creating an HMI application, e.g. for a specific client or server hardware.

In most cases, an HMI application is created without using the real hardware. The project is then simulated with ARsim. The project is expanded step by step, and the HMI application is adapted to the customer's needs.

The "Performance guidelines" chapter lists several factors and considerations that can improve and optimize the performance of an HMI application.

A statement about the performance of an HMI application can best be observed by changing the page.

The duration of the page change depends on several factors:

- Number of widgets on a page
- Which content is exchanged when a page is changed
- How many bindings (e.g. OPC UA) are active on the respective page
- How many event bindings are executed on the respective page or globally



[HMI \ mapp View \ Guides \ mapp View performance guidelines](#)

[HMI \ mapp View \ Guides \ mapp View performance guidelines \ Software guidelines \ Taking into account the number of active bindings](#)

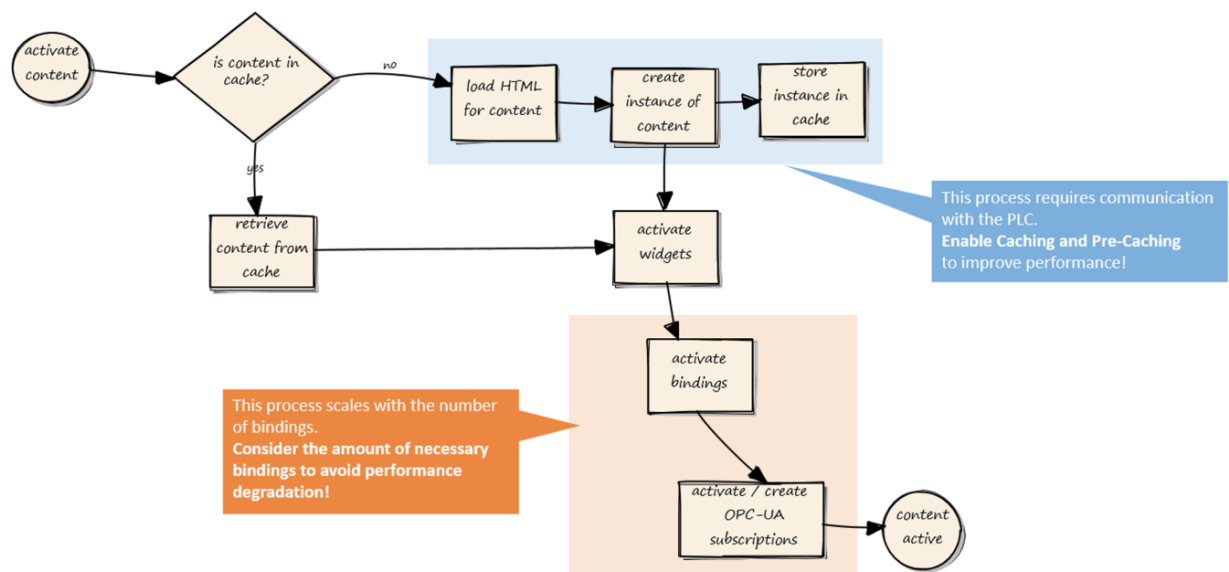


Figure 9: Taking into account the number of active bindings.

Exercise: Analyze the performance of an HMI application

The performance of an HMI application can best be observed during a page change.

- 1) Analyze "MainPage" active bindings / event bindings on the "Clients Info" diagnostics page.
- 2) Switch to the "Info page" and update the client information in the browser.
- 3) Compare with the hardware guidelines in Automation Help.



The data for the respective page can be updated using the "Clients Info" button. The figures for the active bindings and event bindings can be used to evaluate performance problems.

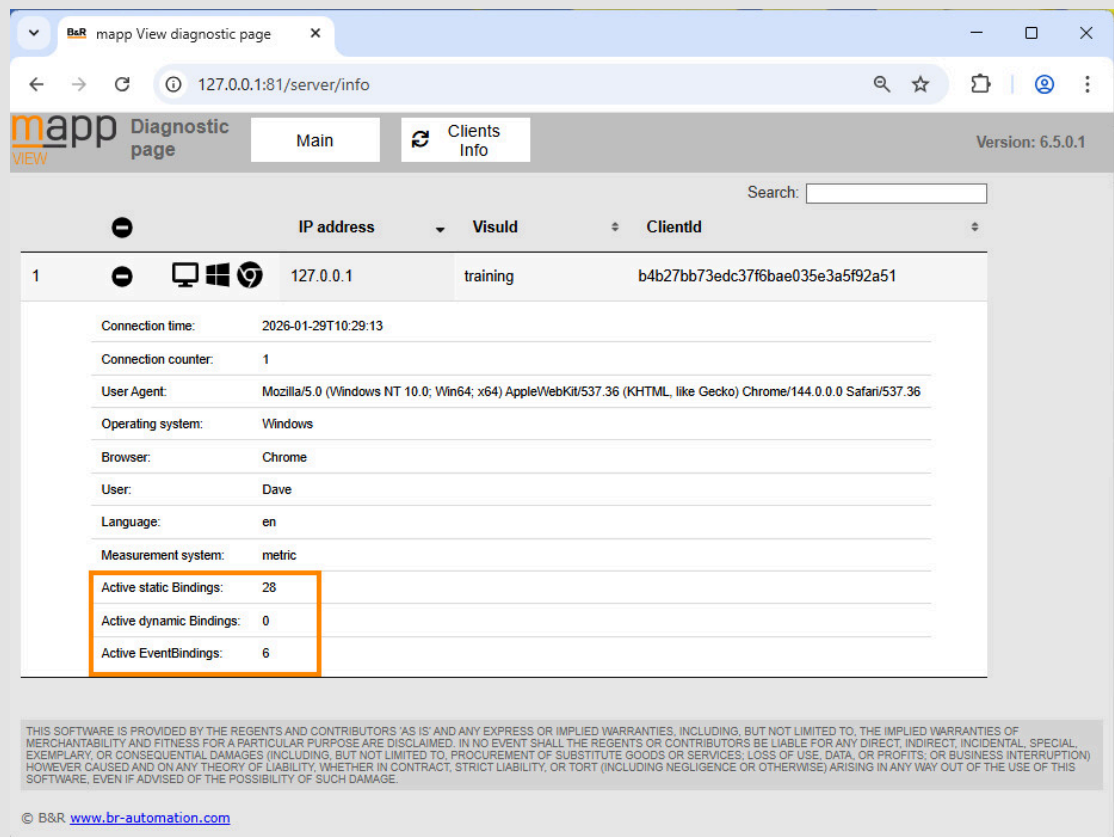


Figure 10: Analysis of bindings and event bindings on the main page.

Three types of bindings are displayed in the client info:

Active static bindings	All bindings (value property of the widgets) that are active with the currently loaded piece of content
Active dynamic bindings	Bindings that are dynamically created and deleted by some widgets at runtime
Active event bindings	All event bindings (reactions to events) that must also be activated with the currently loaded piece of content

Exchanging a piece of content at runtime

In many HMI applications, configuration pages are created on which configuration areas are repeated multiple times.

For example, it is common for injection molding machines to have several heating zones. Depending on the characteristics of such a machine, the configuration of these heating zones then exists 1 - 100 times. To make the HMI application as simple as possible, these configuration areas are copied within a page and, depending on the machine configuration, the setting areas that are not required are hidden (visible binding).

At runtime, such pages are typically relatively slow in terms of screen loading – even if only a few configuration elements are supposedly displayed. The reason is that invisible widgets are also present and active in the background (binding of OPC UA variables). From a performance perspective, it therefore makes no difference whether 100 widgets are visible on a page and 900 widgets are invisible or whether 1000 widgets are visible on the page.



All configured widgets of a piece of content are always supplied with data regardless of their visibility and are also loaded in memory on the client. Resources are also required on the OPC UA server for each active OPC UA variable.

In cases where it is necessary to replicate or dynamically display content – depending on the configuration of a machine – it is recommended to divide these setting areas into several pages. If 100 temperature zones need to be configured, for example, these 100 zones could be divided into 10 pages with 10 zones each.

However, the preferred way would be to use corresponding application logic to dynamically load the pieces of content for the configuration in an area of a page by using action "LoadContentInArea".

[HMI \ mapp View \ Engineering \ Events and actions \ Action \ ClientSystem actions \ LoadContentInArea](#)

Exercise: Reload pieces of content

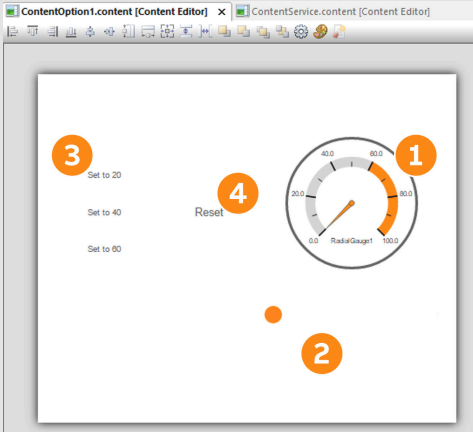
The goal of this exercise is to use a ContentControl widget to load different pieces of content with a different number of bindings and event bindings at runtime and to analyze the active bindings in the "Client info".

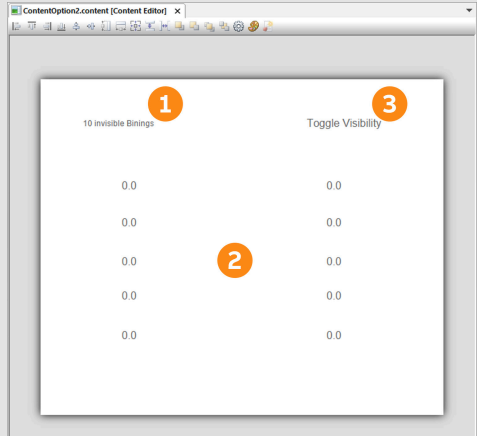
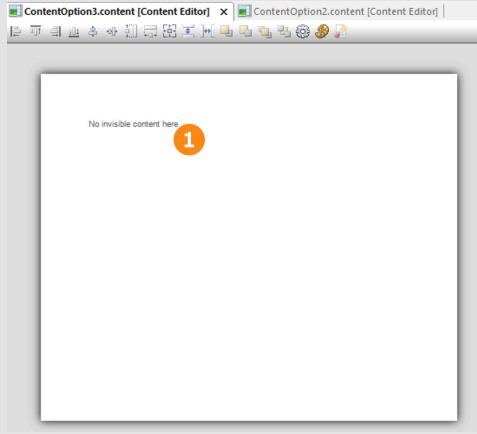
In the first optional piece of content, there should be one binding and four event bindings; in the second optional piece of content, 10 bindings for invisible widgets; the third piece of content will be empty as a comparison.

- 1) In the Logical View under ServicePage, create three new pieces of content ("ContentOption1", "ContentOption2", "ContentOption3") with a width of 600 and a height of 500.
- 2) Add and configure a GroupBox in the "ContentService" of the ServicePage:

Name	Text	backColor	Position	Size
GroupBoxChangeContent	Exchange a piece of content	rgba(255,136,0,0.5)	130;490	600;540

- 3) Add a ContentControl widget to the GroupBox widget and enter "ContentOption1" for property contentId. The piece of content specified here is loaded the first time the widget is loaded.
- 4) Add a ButtonBar with a total of three ToggleButton widgets in "ContentService". The displayed text of the three buttons should be "Load content [no.]".
- 5) Fill the three new pieces of content ("ContentOption1", "ContentOption2", "ContentOption3") with content:

Content	Contents	Result
ContentOption1	1) RadialGauge widget with value binding on (2) 2) Widget BasicSlider 3) Three Label widgets, each with an event binding to the click event that changes the value of the RadialGauge widget 4) Button widget with event binding to a click event, which sets the RadialGauge widget back to 0	 Figure 11: Content ContentOption1

Content	Contents	Result
ContentOption2	1) Label widget for info, without binding 2) 10 NumericOutput widgets that are all invisible but each have a value binding to any OPC UA variable 3) ToggleButton widget with binding to the visible property of all NumericOutput widgets	 The screenshot shows a content editor window titled 'ContentOption2.content [Content Editor]'. It displays a layout with three main components: a label '10 Invisible Bindings' (marked with a red circle 1), a grid of 10 numeric output widgets (marked with a red circle 2), and a toggle button labeled 'Toggle Visibility' (marked with a red circle 3). The numeric output widgets are arranged in two columns of five, each displaying the value '0.0'. Figure 12: Content ContentOption2
ContentOption3	1) Label widget for info, without binding	 The screenshot shows a content editor window titled 'ContentOption3.content [Content Editor]'. It displays a single label widget with the text 'No invisible content here' (marked with a red circle 1). Figure 13: Content ContentOption3

- 6) For the ButtonBar, create an event binding for the "SelectedIndexChanged" event so that for each button (value = 0, 1 or 2) the specified piece of content is loaded into the ContentControl widget.
- 7) Transfer the project and view the number of active bindings and event bindings in the "Clients Info" after loading each piece of content.

**Result:**

The three pieces of content displayed look like this in the HMI application:

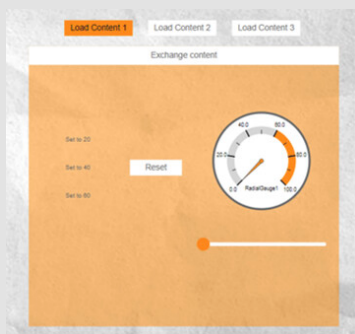


Figure 14: First piece of content loaded

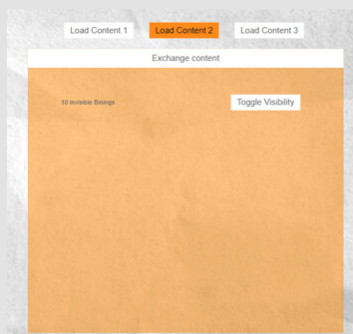


Figure 15: Second piece of content loaded

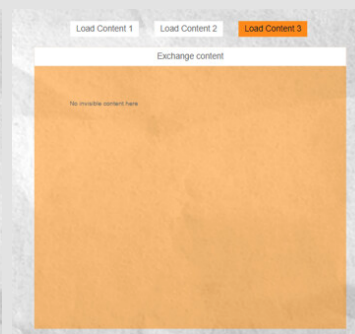


Figure 16: Third piece of content loaded

Although none of the NumericOutput widgets are visible in "ContentOption2", when analyzing the active bindings in the "Client Infos" of the mapp View Diagnosis page, you can see that all bindings of the invisible widgets are active.

mapp

VIEW

Diagnostic page

Main

Clients Info

Version: 6.5.0.1

Search:

		IP address		Visuld		Clientid	
1		127.0.0.1		training		ef33022ae2f7a649b822fb4f4fe432d5	
Connection time: 2026-02-09T15:04:06							
Connection counter: 1							
User Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/144.0.0.0 Safari/537.36 Edg/144.0.0.0							
Operating system: Windows							
Browser: Microsoft Edge							
User: Anonymous							
Language: en							
Measurement system: metric							
Active static Bindings: 30							
Active dynamic Bindings: 0							
Active EventBindings: 4							



[HMI \ mapp View \ Widgets \ System \ ContentControl](#)



In addition to using the ContentControl widget, it is also possible to use the "LoadContentInArea" action via an event binding and exchange the content for an entire area.

3 Displaying diagrams

Diagrams are used for the visual representation of one or more values. For example, the progression of a signal over time is displayed in a two-dimensional coordinate system – similar to an oscilloscope.

Charts display graphs in a widget where the horizontal x-axis (abscissa) represents time, and the vertical y-axis (ordinate) displays the process values.

Charts not only allow you to observe a specific process value; more important, they allow you to track how this value changes over time.

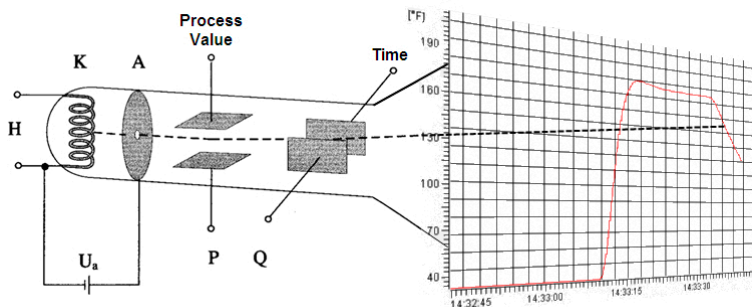


Figure 17: Virtual oscilloscope - Chart widgets

Possibilities of OnlineChartHDA and XYChart:

- Displaying live or historical process values over time
- Displaying the recorded values from the application program
- Displaying multiple graphs differentiated by color
- Measurement cursor for calculating values at intersections
- Flexible configuration of value and time axes
- Zooming in and scrolling through data in the chart
- Showing and hiding curves



[HMI \ mapp View \ Widgets \ Chart](#)



This training module does not cover additional methods of representation such as OnlineChart, LineChart, Timeline, RadialGauge, LinearGauge, BarChart and ProfileGenerator.

3.1 OnlineChartHDA

Widget "OnlineChartHDA" is used to display a live value over time.

This widget visually displays current OPC UA HDA (Historical Data Access) data and online data.



Widget "OnlineChartHDA" requires the "mapp View Premium Widgets" license (1TCMPVIEWWGT.10-01).



[HMI \ mapp View \ Widgets \ Chart \ OnlineChartHDA](#)

3.1.1 Configuring OnlineChartHDA

In this exercise, the OnlineChartHDA widget is used to display process variable "CurrentTemperature" in the mapp View HMI application. Some configurable properties of the widget are used.

Exercise: Display live signal in widget "OnlineChartHDA"

An OnlineChartHDA widget should be added to an additional page in the HMI application

- 1) Add a new page "HDAChartPage" in the Logical View and assign "MyLayout".
- 2) Create a new "ContentHDAChart" for AreaMain on the page.
- 3) Assign "ContentTop", "ContentLeft" and "ContentHDAChart" to the corresponding areas of the page.
- 4) Add an additional button to "ContentLeft" in order to navigate to the page.
- 5) Add a OnlineChartHDA widget to ContentHDAChart.
- 6) Name the widget and under "Collections" \ "Graph" and click on "Data" \ "value" and then assign global process variable "CurrentTemperature".

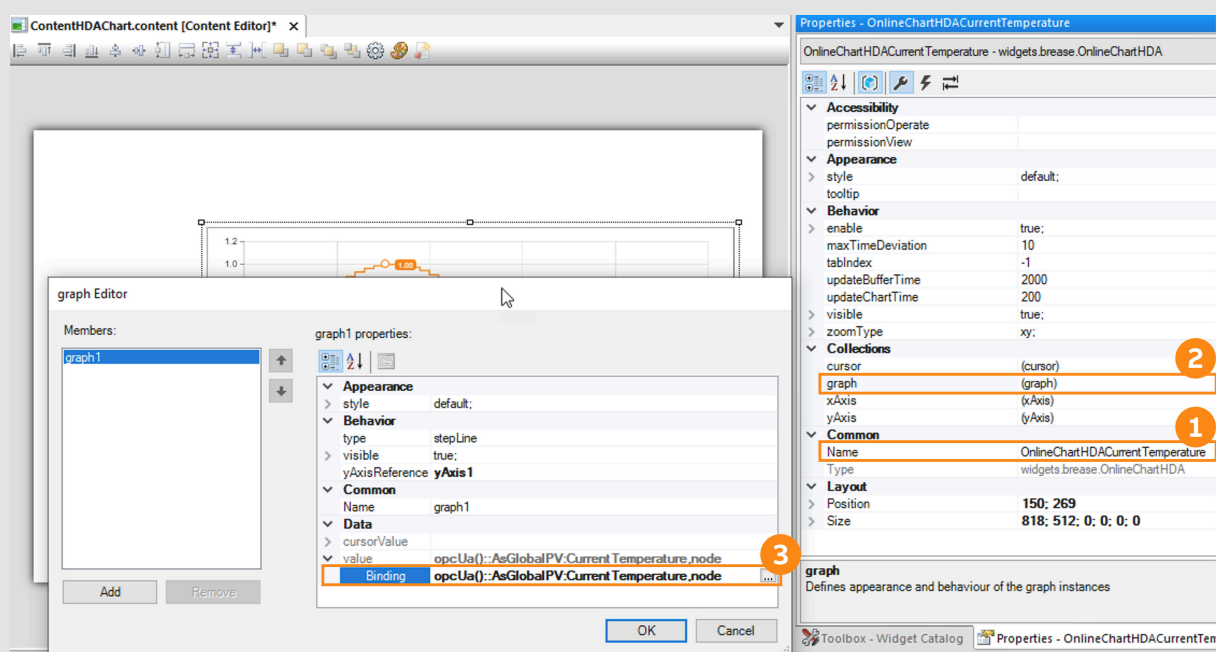


Figure 18: Binding the variable "CurrentTemperature" to the graph.

- 7) Transfer the project.
- 8) Change the temperature setpoint on the main page in the HMI application.
- 9) Switch to the HDAChartPage in the HMI application.



It can be observed that the value change is not displayed before the page change [1]. The same applies when resetting the setpoint temperature to the default value=35 [3].

- 1 - Change temperature setpoint
- 2 - View page change and value change
- 3 - Change temperature setpoint
- 4 - Page change

This means that without recording the value changes from the OPC UA server, data changes are only displayed if the content is active.

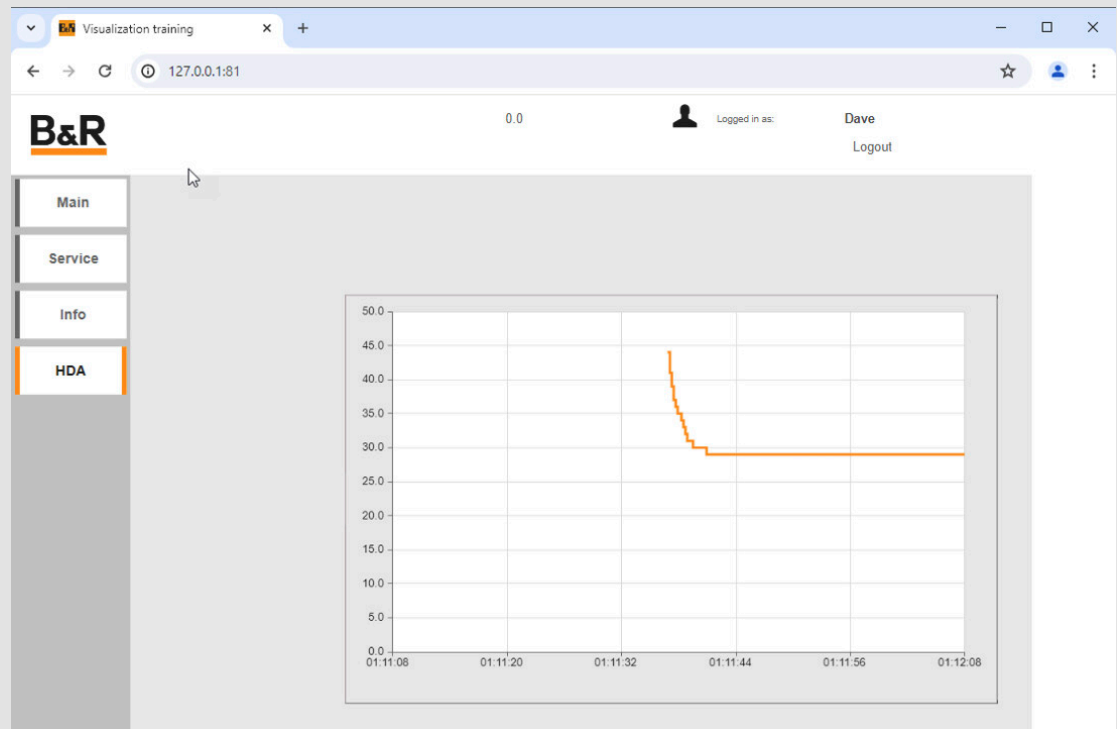


Figure 19: Widget "OnlineChartHDA" without historizing

Exercise: Enable historical records

The goal is for value changes to be recorded even if the HDACHartPage is not displayed.

To achieve this, the OPC UA variable that is displayed in the OnlineChartHDA widget must be configured accordingly; the historizing, sampling rate and ring buffer must be configured.

- 1) Open the OPC UA default view (.uad).
- 2) Enable historizing for the global variable "CurrentTemperature".
- 3) Set the sampling time to 200 ms.
See OPC UA sampling, timer interval.
- 4) Set the buffer size / recording duration to 10 min.
Calculate the buffer size.



It can be observed that the value change is completely recorded and loaded with the page change.

- 1 - Change temperature setpoint
- 2 - Change temperature setpoint
- 3 - View page change and value change

This means that by recording the value changes from the OPC UA server, data changes are displayed correctly.

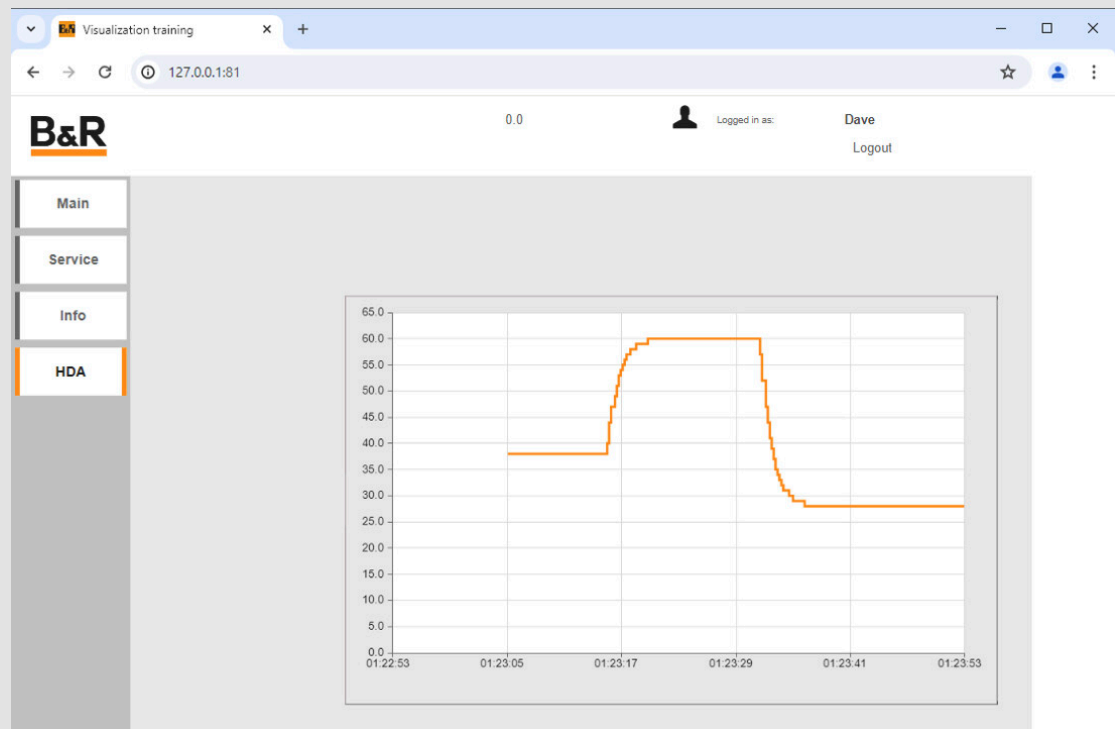


Figure 20: Widget "OnlineChartHDA" with historizing



By default, the values in the OnlineChartHDA are read from the OPC UA server in the set **UpdateBufferTime**; the update on the widget takes place in the set **UpdateChartTime**. Both times can be configured in the properties of the widget.

3.1.2 Analyzing historical data in widget OnlineChartHDA

Exercise: Start and stop the update in the widget

To be able to analyze data on the client, the update must be paused. This is done in the event binding via an action of the OnlineChartHDA widget.

- 1) Add two buttons to start (Unfreeze) and stop (Freeze) the display.
- 2) Configure the click event and the corresponding widget action for each button.

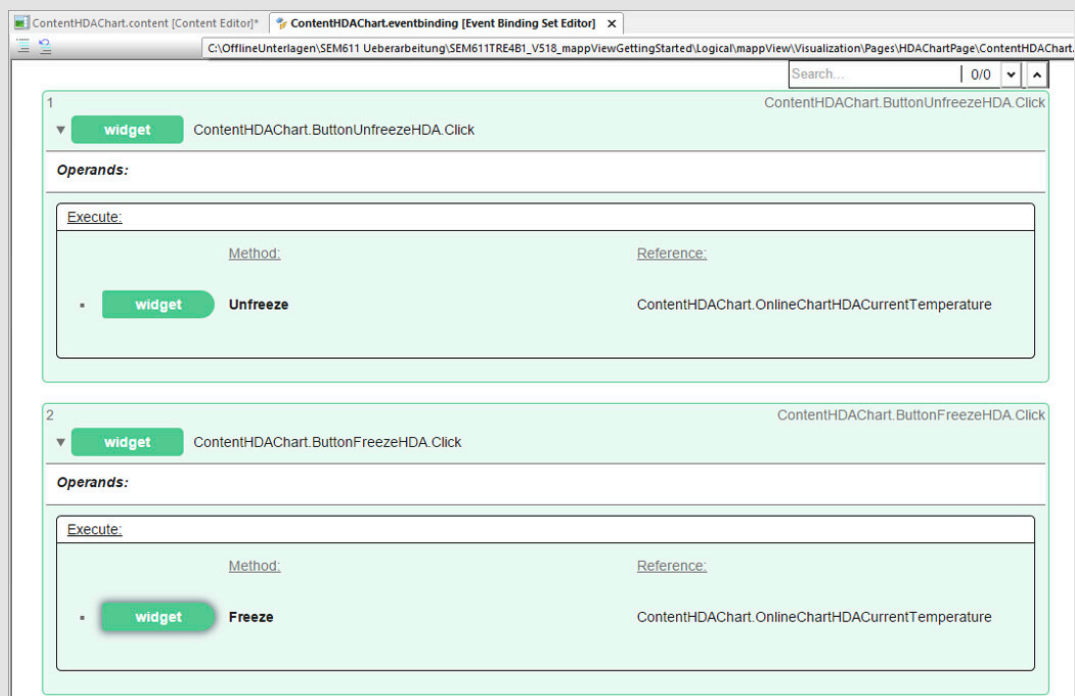


Figure 21: Event binding to start and stop the update in OnlineChartHDA



In freeze mode, the data can be analyzed using the mouse or touch screen.

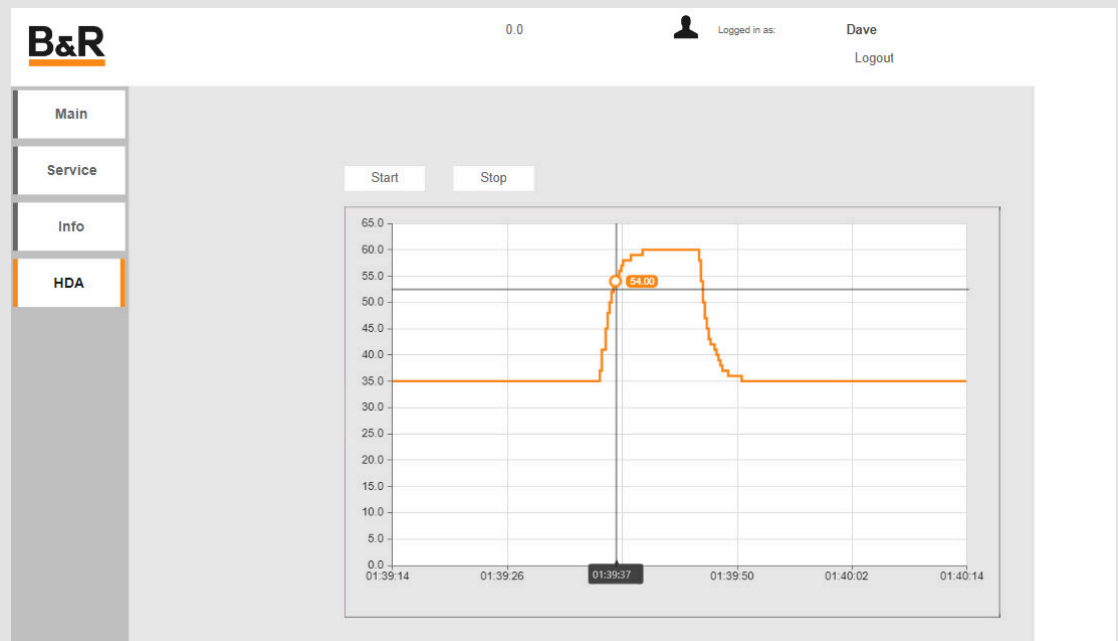


Figure 22: Data logging analysis

Additional actions and events are available to operate the "OnlineChartHDA" widget.

- Scrolling in the visible area
- Scrolling the axis
- Zooming functions
- Setting a start and end time in freeze mode
- Setting the visible time range



[HMI \ mapp View \ Widgets \ Chart \ OnlineChartHDA \ Actions and events](#)

3.1.3 Configuring a range of values for the y-axis

The range of values of a y-axis can be configured in the widget according to the requirements via the "rangeMode" property of the axis.

The following options are available:

- **autoscale:** The widget automatically adjusts the range of values based on the minimum and maximum values.
- **fromConfiguration:** The values of min and max are used.
- **fromSource:** The limits of the OPC UA variables bound to a graph are used.

Exercise: Change the range of values of the y-axis to "fromSource"

- 1) Configure the limits Low=20 and High=60 for the OPC UA variable "CurrentTemperature".
- 2) Set the "rangeMode" property of widget OnlineChartHDA for the Y-axis to "fromSource".

3.2 XYChart

The XYChart widget is used to display numerical data pairs in a chart. The XYChart widget is used to display an array of values (e.g. temperatures) over another array of values (e.g. pressure).



Widget "XYChart" requires the "mapp View Premium Widgets" license (1TCMPVIEWWGT.10-01).



[HMI \ mapp View \ Widgets \ Chart \ XYChart](#)

3.2.1 Configuring widget XYChart

Optional exercise: Display the motor characteristic curve in widget XYChart

In this exercise, widget "XYChart" is used to display the torque of the process variable "::ChartData.Torque" above the speed of the process variable "::ChartData.Speed" in the mapp View HMI application.

- 1) Enable all variables of the program "ChartData" in the OPC UA default view with "AutomaticEnable -> Recursive".
- 2) Assign units from the Toolbar: Set Current [A], Speed [rpm], Torque [Nm], PeakCurrent [A] and PeakTorque [Nm] to the OPC UA values.
- 3) Add new page "XYChartPage", assign a layout and create new piece of content "ContentXYChart".
- 4) Add an XYChart widget on the new piece of content.
- 5) Click "Collections" \ "graph" and assign the process variable "::ChartData.Speed" to "Data" \ "valueX".
- 6) Click "Collections" \ "graph" and assign the process variable "::ChartData.Torque" to "Data" \ "valueY".
- 7) Click "Collections" \ "xAxis" and assign the unit [rpm] to "Appearance" \ "unit" for each system of measurement.
- 8) Click "Collections" \ "yAxis" and assign the unit [Nm] to "Appearance" \ "unit" for each system of measurement.



Widget "XYChart" displays the motor characteristic curve, which is composed of the torque and speed.

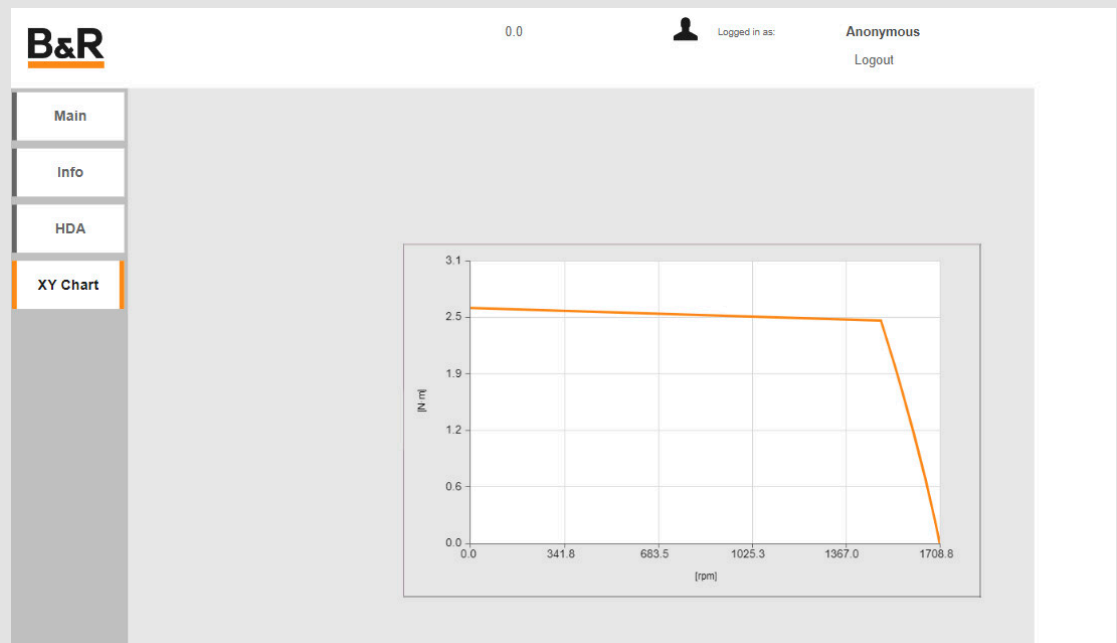


Figure 23: Motor characteristic curve: Torque in relation to speed

Optional exercise: Display peak torque in a second graph

A second graph that represents the peak torque of the process variable "::ChartData.PeakTorque" should be displayed in widget "XYChart".

- 1) Click "Collections" \ "graph" and add a second graph.
- 2) Change to "1px" under Appearance \ "lineWidth".
- 3) Add the process variable "::ChartData.Speed" under "Data" \ "valueX".
- 4) Add the process variable "::ChartData.PeakTorque" under "Data" \ "valueY".



The second graph is configured to display the motor characteristic curve, which is composed of the peak torque and speed.

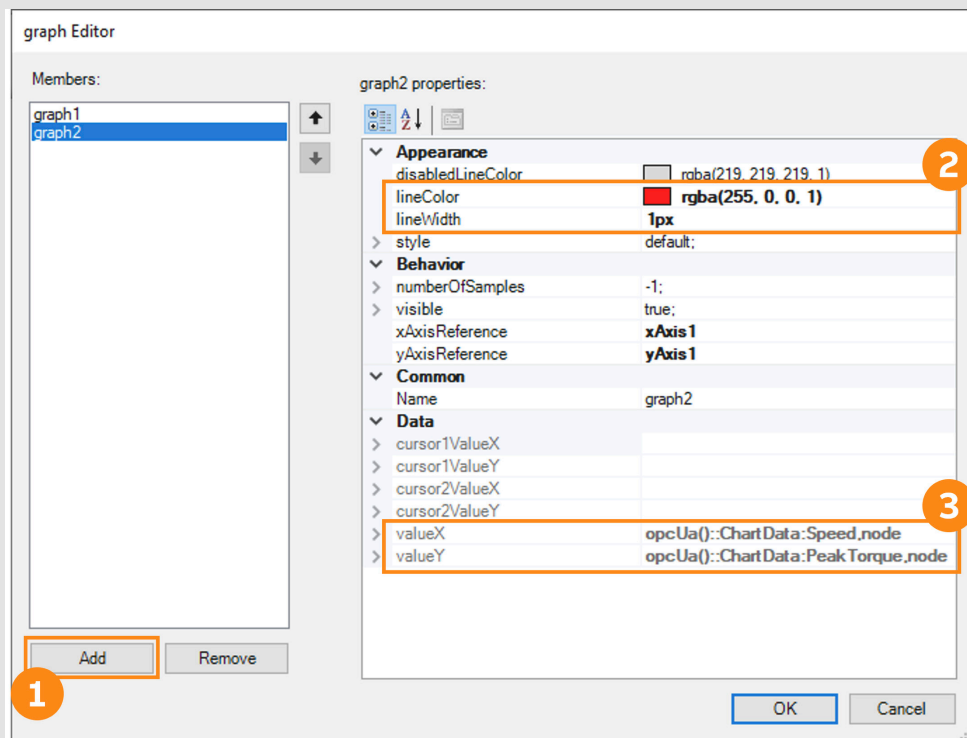


Figure 24: Adding a second graph

Optional exercise: Display the current and peak current on a second y-axis

In widget "XYChart", two additional graphs should be displayed on a second y-axis to show current "::ChartData.Current" and peak current "::ChartData.PeakCurrent".

- 1) Click "Collections" \ "yAxis" and add a second axis.
- 2) Under "Appearance" \ "unit", assign the unit [A] for each measurement system.
- 3) Under "Appearance" \ "position", select the option "right".

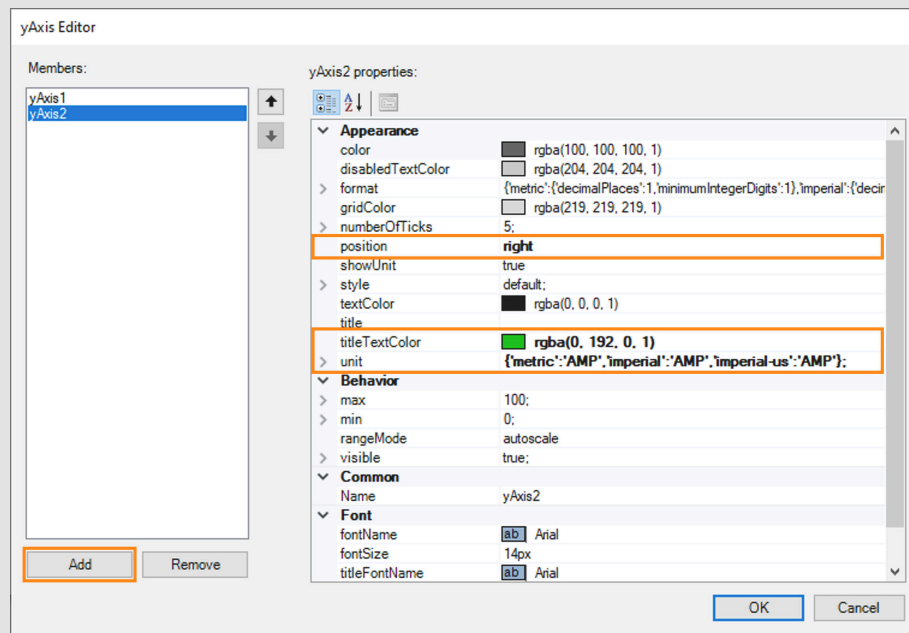


Figure 25: A second y-axis has been added to widget "XYChart".

- 4) Click "Collections" \ "graph" and add two new graphs.
 - Add the process variable "::ChartData.Speed" under "Data" \ "valueX".
 - Under "Data" \ "valueY", assign the process variables "::ChartData.Current" and "::ChartData.PeakCurrent".
- 5) For both graphs, the option "yAxis2" is selected under "Behavior" \ "yAxisReference".

3.2.2 Analyzing the data in the XYChart

By default, the values of the graphs in widget "XYChart" are received from the OPC UA server at the RefreshRate used in the mapp View configuration (default, slow, fast). The update on the widget takes place in the configured **Update-ChartTime**.

Optional exercise: Start and stop the update in widget XYChart

To be able to analyze data on the client, the update must be paused. This is done in the event binding via an action of the XYChart widget.

- 1) Add two buttons to start (Unfreeze) and stop (Freeze) the display.
- 2) Configure the click event and the corresponding widget action for each button.



In freeze mode, the data can be analyzed using the mouse or touch screen.

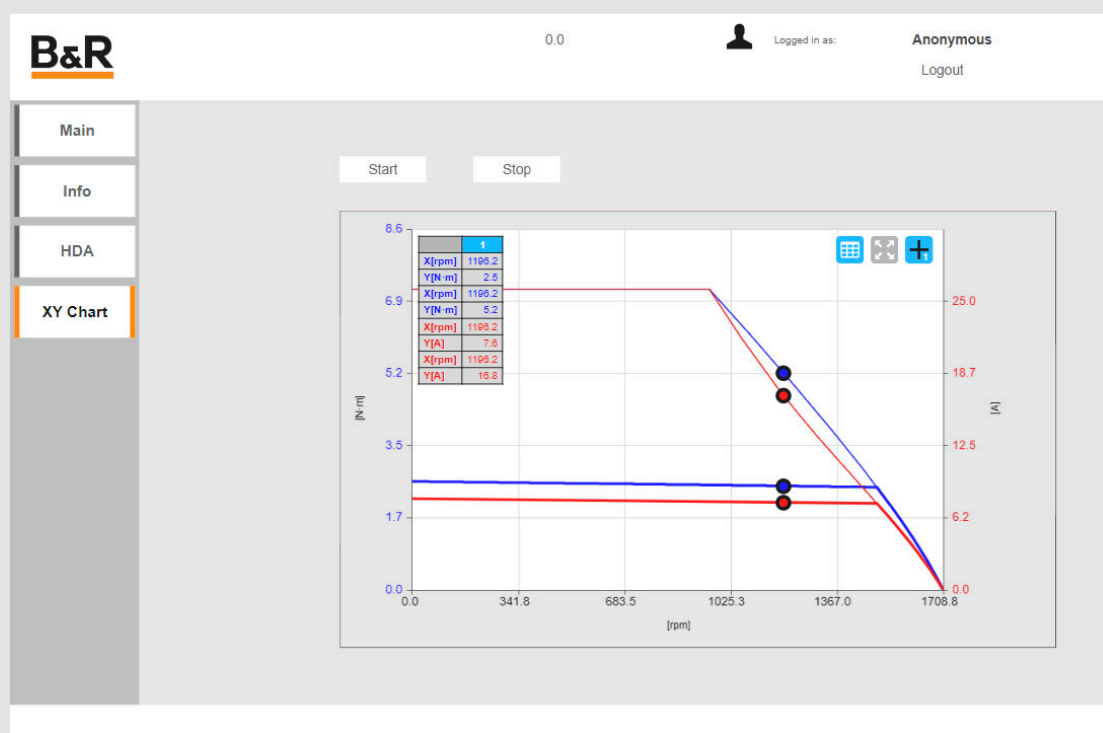


Figure 26: Analysis of the recording

Additional actions and events are available to operate widget XYChart.

- Scrolling in the visible area
- Scrolling the axis
- Zooming functions
- Updating the display in freeze mode



[HMI \ mapp View \ Widgets \ Chart \ XYChart \ Actions and events](#)

4 Customized widgets

Customized widgets allow faster configuration of frequently used widgets of the same type.

There are 3 types of customized widgets:

- **Derived widgets:** To reuse a widget with identical properties that has already been configured in the HMI application.
- **Compound widgets:** A widget group that is used multiple times in the HMI application.
- **Custom keyboards:** Numeric or alphanumeric virtual keyboard with user-defined size and arrangement of keys.

These types of widgets are managed in the Logical View as a widget library in the package "mappView" \ "Widgets".



[HMI \ mapp View \ Engineering \ Customized widgets](#)

4.1 Widget library

A widget library is used to add customized widgets that you have created yourself. The widget library also defines the namespace of the widgets it contains.



After adding a customized widget to a widget library, it must be compiled for use in the Widget Catalog and on the runtime system. The widget library can be compiled under "Project" \ "Build Widget Library".



[HMI \ mapp View \ Engineering \ Customized widgets \ Widget library](#)

[HMI \ mapp View \ Engineering \ Customized widgets \ Widget Library \ Building a widget library](#)

Exercise: Add a widget library

The goal of this task is to create your own widget library. This should have the namespace "Training".

- 1) Add a new widget library from the Toolbox under "mapp View" \ "Widgets".
- 2) Change the package name from "WidgetLib" to "Training".
- 3) Change the ID of the file "Description.widgetlibrary" to "Training".



The widget library is now created, which results in the project structure shown.

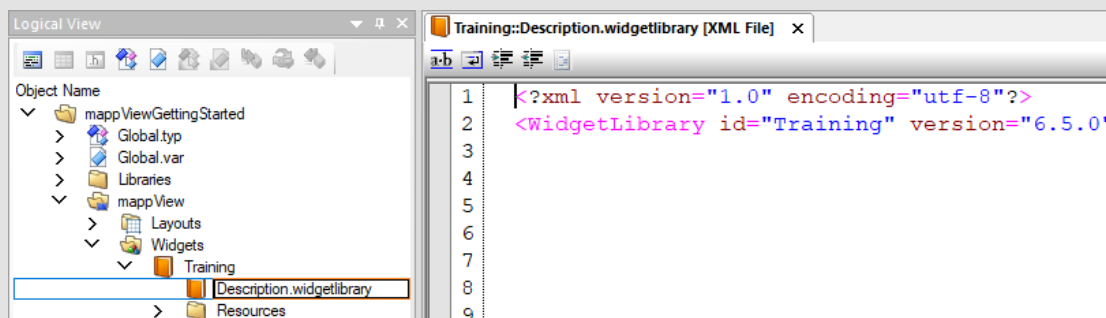


Figure 28: Widget library namespace

4.2 Derived widgets

A derived widget is created in the Content Editor on the basis of a widget whose configuration is already "finished". When creating a derived widget, you can assign your own widget name, which is unique due to the namespace of the selected widget library.

In order to reuse the widget in a piece of content, the widget library must first be compiled.



[HMI \ mapp View \ Engineering \ Customized widgets \ Derived widget](#)



Subsequent changes to a derived widget affect all instances.

Exercise: Create a derived widget

The goal of this task is to reuse the NumericInput widget on the MainPage. This widget will be the basis for a new customized Derived Widget.

- 1) Select the NumericInput widget "NumericInputSetTempValue", which has units already configured, for piece of content "ContentMainPage".
- 2) Use the Content Editor toolbar to create a derived widget.
- 3) Enter the name "NI_Celsius" in the field "Widget Typename".

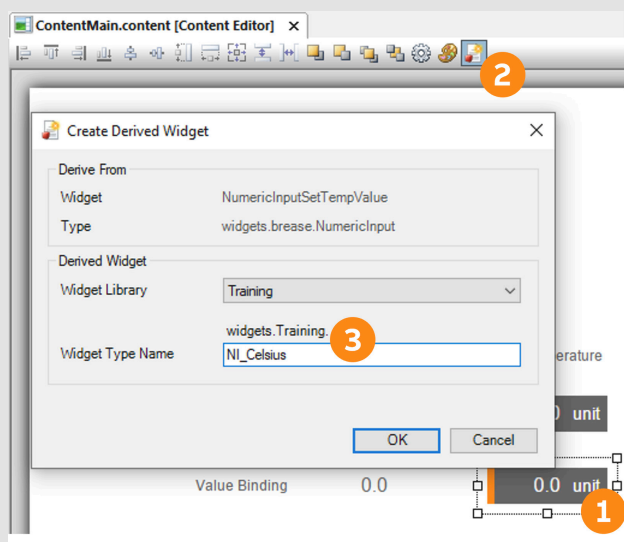


Figure 29: Creating a derived widget based on a "NumericInput" widget

- 4) Compile the widget library (e.g. with SHIFT + F7 or under Project \ Build widget library).



After compiling the widget library, widget "NI_Celsius" can be taken from the Object Catalog and added to the piece of content.

The derived widget has the same properties as the base widget – or in this case, as a NumericInput widget with units configured.

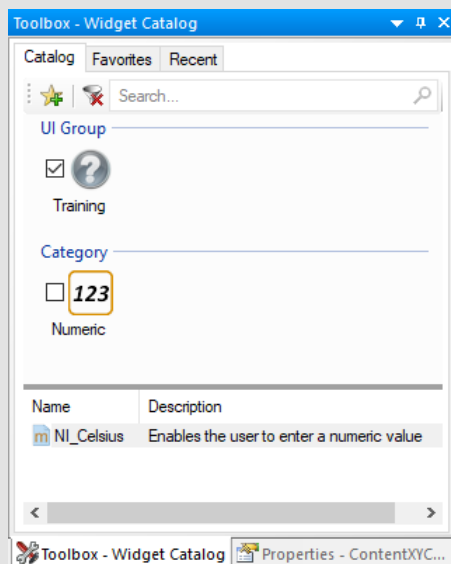


Figure 30: Widget "NI_Celsius" can be added to any piece of content.

4.3 Compound widget

Compound widgets allow existing widgets to be combined into a user-defined widget. In addition, the interface can be defined for use in a piece of content. The wiring between the compound widget property and a child widget takes place within the compound widget.

Use cases:

- Reusing identical groups of widgets
- Easier engineering of properties
- Internal logic or functionality of a compound widget

4.3.1 Create empty compound widget

Unlike a derived widget, which is created from a piece of content, a compound widget is taken from the Object Catalog and added to a widget library.

The following exercises describe in several steps how to create a "Motor" widget using Automation Help.

This widget allows control of a simple motor.

- Starting the motor with a "PushButton" widget
- Stopping the motor with a "PushButton" widget
- Setting the maximum permitted speed with a "NumericInput" widget
- Displaying the current motor speed with a "RadialGauge" widget

These 4 widgets are part of the compound widget called "Motor". In addition, it should be possible to bind a structure to the widget that contains all relevant OPC UA variables for controller of the motor.

This structure is already included as an array in the "mappViewGettingStarted" project and allows the simultaneous control of up to 10 motors.

Exercise: Enable and configure motor variables in the OPC UA default view

The structure array "Motor" must be enabled in the OPC UA default view and the individual array elements must be configured.

- 1) In the "Motor" program, enable the "Motor" array as an OPC UA variable by right-clicking and selecting "AutoEnable -> Recursive".
- 2) In the OPC UA properties, select the option "Show array elements".
- 3) Configure the **speedLimit** member of each array element with an EU range of 0-1000.
- 4) Configure the **speedLimit** member of each array element with an "rpm" engineering unit.
- 5) Configure the **motorSpeed** member of each array element with an EU range of 0-1000.
- 6) Configure the **motorSpeed** member of each array element with an engineering unit "rpm".



The OPC UA variables can be configured collectively (via "Common properties").

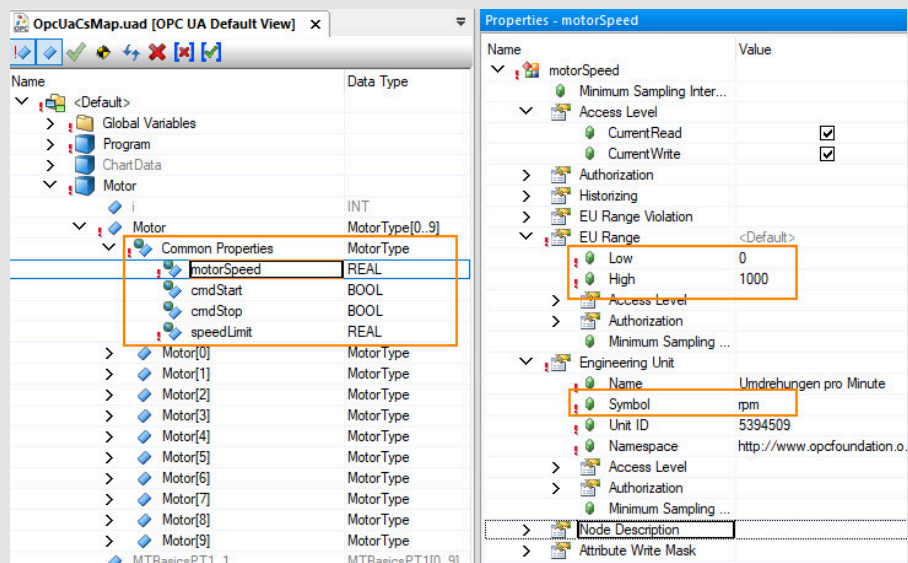


Figure 31: Configuration of the structure array "Motor" in the OPC UA default view

Exercise: Add a new compound widget

The goal of this task is to create a new compound widget. This forms the framework for further widgets and is added to the existing "Training" widget library. The properties of the compound widget are edited using the XML editor.

- 1) Select the widget library "Training" and add a new compound widget from the Toolbox.
- 2) Rename the created package to "Motor".
- 3) Open the file "Widget.compoundwidget".
- 4) Change the ID from "compoundwidget_0" to "Motor".
- 5) Set property "width" to 300 and property "height" to 350.

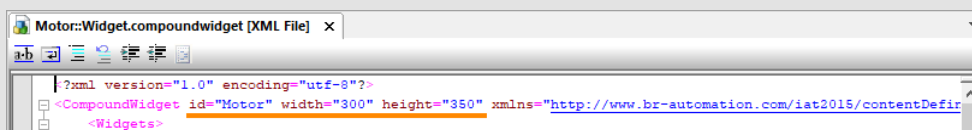


Figure 32: Changing properties of the compound widget



After compiling the widget library using SHIFT + F7, the new compound widget can be taken from the Widget Catalog and added to the piece of content.

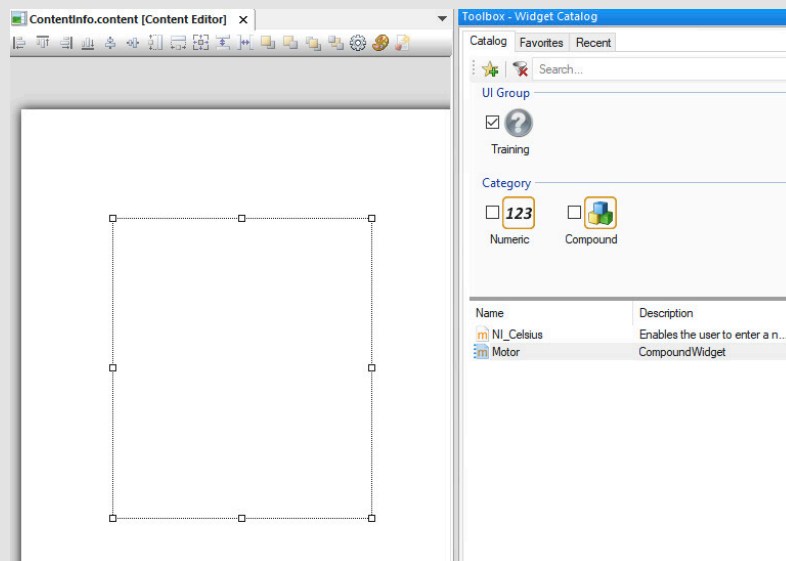


Figure 33: Widget "Motor" is added to a piece of content.



If a compound widget is changed, the piece of content that the compound widget is located on must be closed and reopened.

4.3.2 Adding widgets to a CompoundWidget

Widgets in a compound widget can be configured either directly in the <Widgets> element of the XML editor or in a separate piece of content, and then transferred to the XML.

Exercise: Add widgets to the compound widget

The next step is to add 5 widgets to the compound widget.

- 1) Add 1 "RadialGauge" widget, 1 "Label" widget, 1 "NumericInput" widget and 2 "PushButton" widgets.



These steps should be carried out independently via Automation Help, see:

[HMI \ mapp View \ Engineering \ Customized widgets \ Compound widget \ Creating compound widgets in the Content Editor](#)

- Create a piece of content and configure the widgets.
- Open the piece of content in the XML editor.
- Copy the widgets from the piece of content to the compound widget.

Widget	Property	Value
RadialGauge	Name (ID)	RadialGaugeRPM
	Position (top, left)	25; 50
	ShowUnit	True
	Unit	RPM
	Format	0 decimalPlaces
Label	Name (ID)	LabelLimit
	Position (top, left)	240; 40

Widget	Property	Value
NumericInput	Text	Limit
	Name (ID)	NumericInputLimit
	Position (top, left)	240; 140
	Size (width, height)	120; 30
	Unit	RPM
	UnitAlign	Right
PushButton	Format	0 decimalPlaces
	Name (ID)	PushButtonStart
	Position (top, left)	300; 40
PushButton	Text	Start
	Name (ID)	PushButtonStop
	Position (top, left)	300; 160
PushButton	Text	Stop



After the Automation Help steps have been carried out, the widget library must be recompiled (SHIFT + F7). The compound widget with its properties is then displayed correctly in the added piece of content.

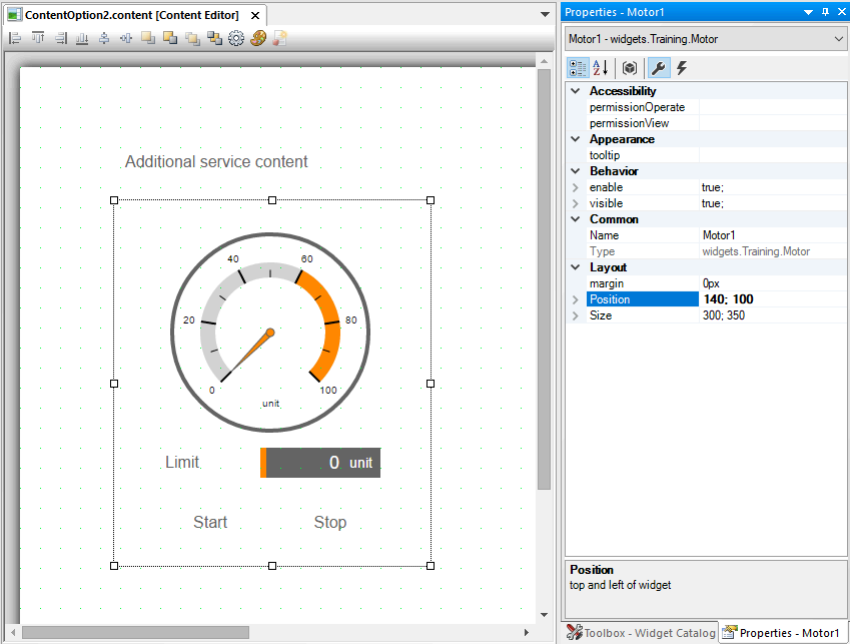


Figure 34: Complete compound widget

4.3.3 Properties of the compound widget

A bindable property of a compound widget is configured in the <Properties> element of the ".compoundwidget" file. A bindable property allows you to bind a variable in a piece of content when configuring the compound widget.

The following options are available:

- User-defined property with value binding
- User-defined property with node binding
- Transmitting a user-defined property to multiple widgets
- User-defined property with a structure binding
- Local properties (private and public)



[HMI \ mapp View \ Engineering \ Customized widgets \ Compound widget \ Compound widgets reference \(XML\) \ Bindable properties](#)

Exercise: Create the "value" property for the MotorWidget, which allows structure binding

In this task, a "value" property is added to the compound widget that has been created. This enables a binding to a structure variable. In the mapping, the corresponding members of the structure are linked to the corresponding properties of the child widget properties present in the compound widget.

- 1) These steps should be carried out independently via Automation Help:



[HMI \ mapp View \ Engineering \ Customized widgets \ Compound widget \ Compound widgets reference \(XML\) \ Bindable properties \ Structure binding of a user-defined property](#)

- 2) Compile a widget library.
- 3) Then open ContentInfo where the compound widget is located.
- 4) Click on "Data" \ "Value" \ "Binding".

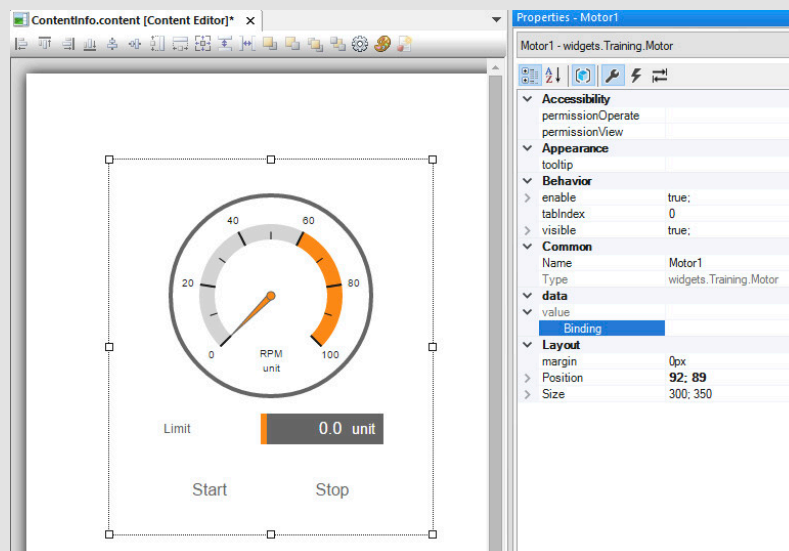


Figure 35: New property "Value"

5) Select the variable, e.g. Motor[0], in the "OPC UA" tab of the variable selection dialog box.

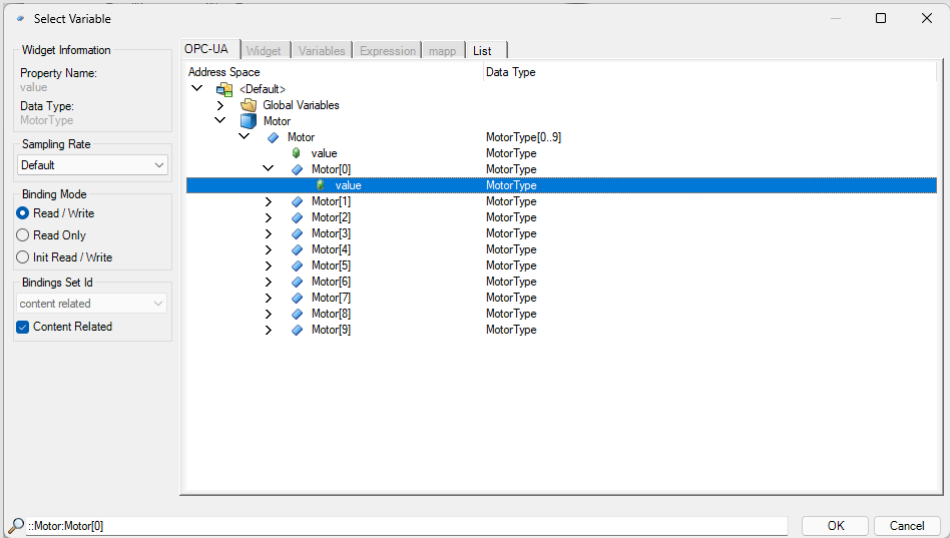


Figure 36: Binding a structure to a compound widget



The compound widget is displayed correctly, but the functionality can be improved with the next exercises.

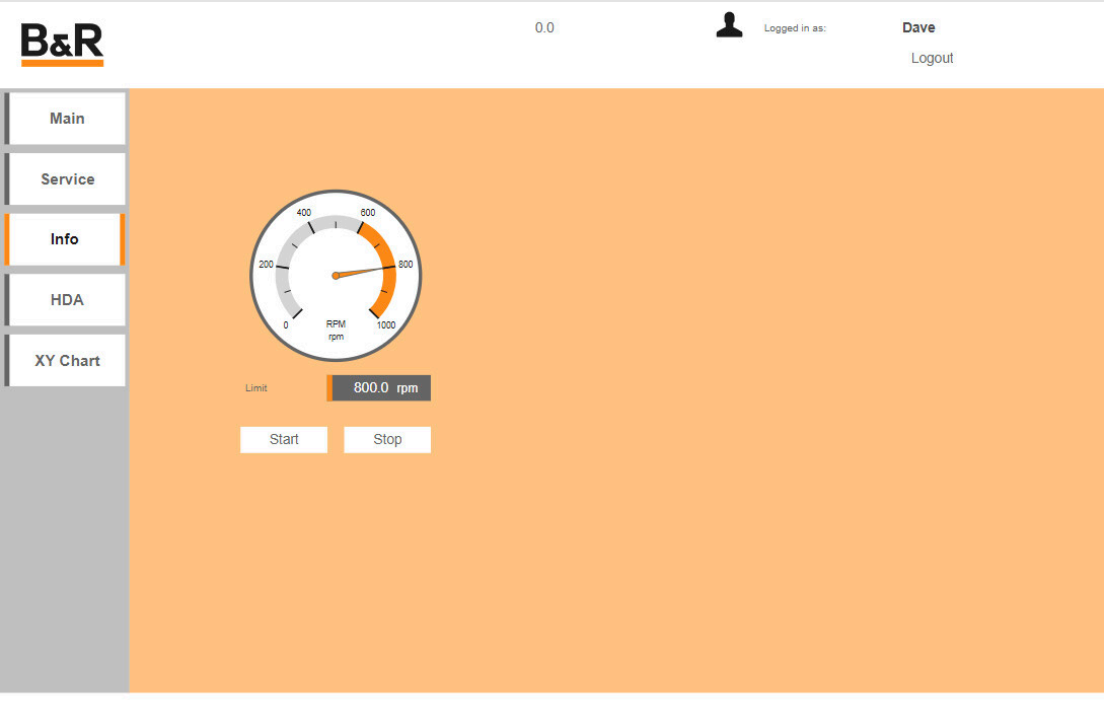


Figure 37: Compound widget at runtime

4.3.4 Events and actions of a compound widget

Events and actions within a compound widget can be configured so that the behavior of a compound widget can be customized more individually.

- An event describes a change of state within a compound widget.
- An action describes an executable operation within a compound widget.

It is also possible to define events and actions for a compound widget, which can be used when configuring a widget instance in the event binding (e.g. trigger a widget's click event within a compound widget in the widget instance).



Visualization \ mapp View \ Engineering \ Customized widgets \ Compound widget \ Compound widgets reference (XML) \

- [Events and actions \(XML\)](#)
- [Compound widget events](#)
- [Compound widget actions](#)

Exercise: Control the enable behavior of "Start" and "Stop" in the compound widget

In this task, the enable status of the Start and Stop PushButton widgets is controlled via an internal EventBinding. If the motor is at a standstill, only the Start button can be pressed; the Stop button is disabled. If the motor is running, only the Stop button can be pressed; the Start button is disabled. The "enable" property and an event binding must be configured for this.

- 1) Open the source code from the compound widget motor.
- 2) Set the "enable" property of the "PushButton" widget with the id="PushButtonStop" to "false".



Figure 38: Changing the default behavior of PushButtonStop

- 3) In the event binding, react to the click event of "Start" and "Stop" and perform the action "SetEnable".

```
<EventBinding>
  <Source xsi:type="widget.Event" widgetRefId="PushButtonStart" event="Click" />
  <EventHandler>
    <Parallel>
      <Action>
        <Target xsi:type="widget.Action" widgetRefId="PushButtonStop">
          <Method name="SetEnable" value="true" />
        </Target>
      </Action>
      <Action>
        <Target xsi:type="widget.Action" widgetRefId="PushButtonStart">
          <Method name="SetEnable" value="false" />
        </Target>
      </Action>
    </Parallel>
  </EventHandler>
</EventBinding>
<EventBinding>
  <Source xsi:type="widget.Event" widgetRefId="PushButtonStop" event="Click" />
  <EventHandler>
    <Parallel>
      <Action>
```

```

        <Target xsi:type="widget.Action" widgetRefId="PushButtonStop">
            <Method name="SetEnable" value="false" />
        </Target>
    </Action>
    <Action>
        <Target xsi:type="widget.Action" widgetRefId="PushButtonStart">
            <Method name="SetEnable" value="true" />
        </Target>
    </Action>
</Parallel>
</EventHandler>
</EventBinding>

```



As a result of this exercise, it can be observed that the behavior of the disabled state of widget "Push-Button" only works if the motor is not running when the mapp View HMI application is started. In order to represent this behavior correctly, the compound widget must be informed of the state of the motor when the piece of content is enabled. Using a compound widget action, the state (e.g. in the "ContentLoaded" event of the piece of content on which the compound widgets are located) can be queried and communicated to each instance of the compound widget.

Exercise: Create the compound widget action "SetButtonEnable"

In the "Motor" compound widget, the Start button is also initially disabled and a "SetButtonEnable" action is created. When called, the action switches the "enable" property of both PushButton widgets according to the current status of the motor.

- 1) Open the source code from the compound widget motor.
- 2) Set the "enable" property of the "PushButton" widget with id="PushButtonStart" to "false".
- 3) Configure the "SetButtonEnable" action.
This action sets the enabled status of both PushButton widgets in the event binding.

```

<Action name="SetButtonEnable">
    <Description>Set init state of PushButton widgets.</Description>
    <Arguments>
        <Argument name="StartEnable" type="Boolean"/>
        <Argument name="StopEnable" type="Boolean"/>
    </Arguments>
    <Mappings>
        <Mapping widget="PushButtonStart" action="SetEnable">
            <Arguments>
                <Argument name="StartEnable" mapTo="value"/>
            </Arguments>
        </Mapping>
        <Mapping widget="PushButtonStop" action="SetEnable">
            <Arguments>
                <Argument name="StopEnable" mapTo="value"/>
            </Arguments>
        </Mapping>
    </Mappings>
</Action>

```

4) Create a new event binding (clientSystem.Event, opcUa.Operand, widgets.Action).

The "SetButtonEnable" action must be called in the ContentLoaded event of the piece of content on which the compound widgets are used for each widget instance.

Each time the piece of content is enabled, the value of the OPC UA variable Motor[instance].motorSpeed is read via an operand and evaluated in the conditions of the event binding with the contentId.

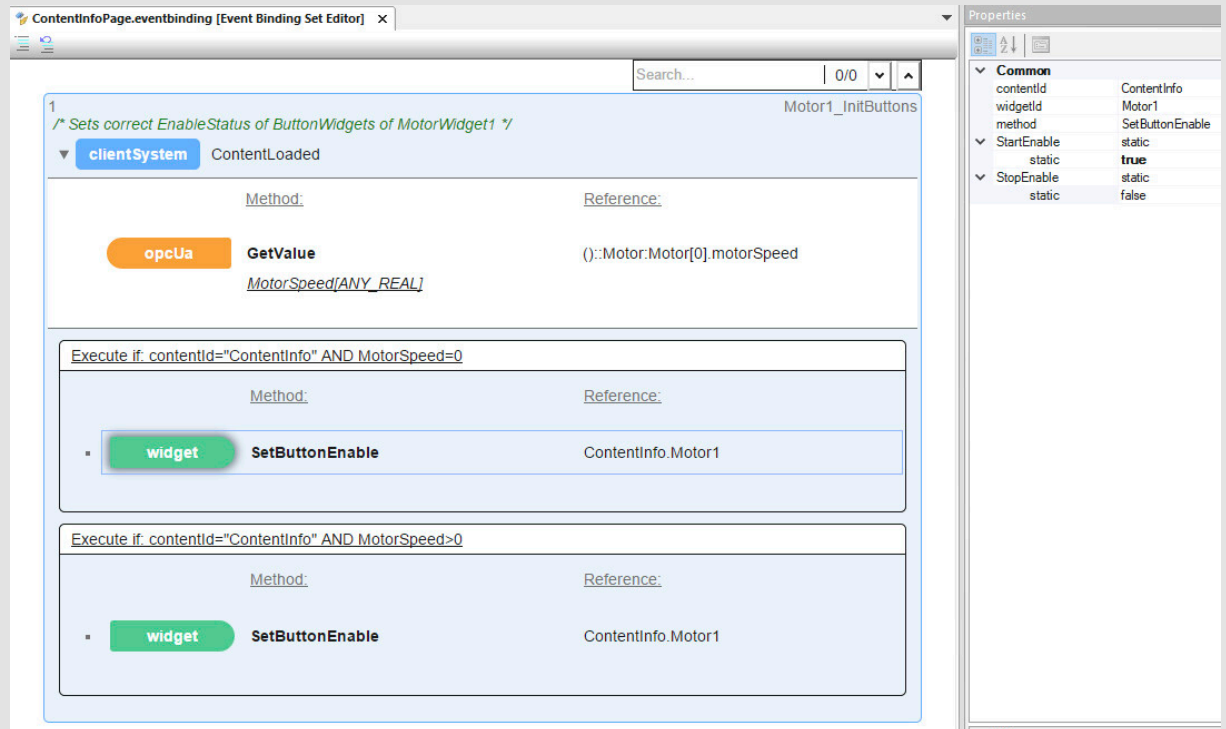


Figure 39: Controlling the disabled state via an event binding

The main goal when creating compound widgets is to reuse your own grouped widgets. Any number of instances of a compound widget can be added to the HMI application.

Exercise: Reuse compound widget "Motor"

Another instance of the "Motor" widget should be configured. The value binding should be made to the next element `Motor[1]` of the structure array, and an event binding similar to the previous task must also be created for the new widget instance.



Here, the element "Motor[1]" of the structure array is used for the second compound widget.

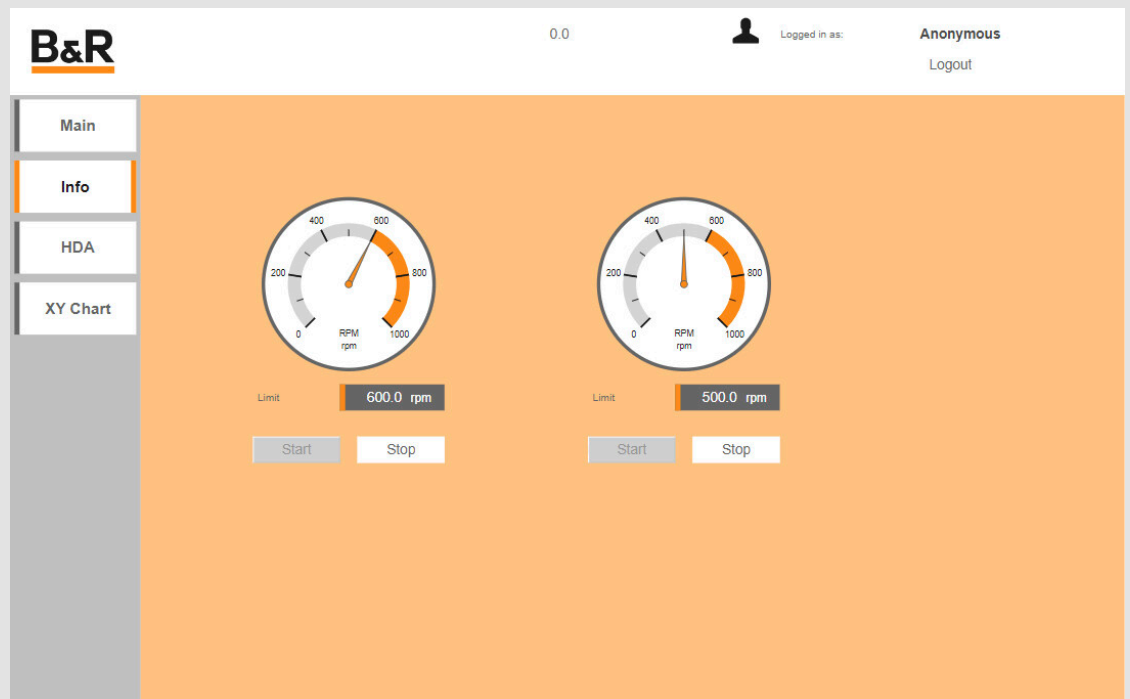


Figure 40: The compound widget can be easily reused.

4.4 Custom keyboards

A numeric keyboard (mapp View 5.8) or alphanumeric keyboard (mapp View 5.10) can be created and used in a HMI application instead of the keyboards provided by mapp View. Individual keyboards can be created for each language for the alphanumeric keyboard.



[HMI \ mapp View \ Engineering \ Customized widgets \ Custom keyboards](#)

5 Summary

The exercises of this training module are designed to deepen participants' basic knowledge of mapp View.

Diagnostic access plays an important role for service and maintenance. The diagnostics page allows the display of mapp View server relevant information in a browser. All data is displayed clearly.

The analysis of data can be ideally displayed using Chart widgets. Live data can be stopped by certain events and actions and thus viewed in more detail.

The individual customizations are ideal for specific customer requests. The compound widget in particular enables faster development and leaves more time for innovation.

Automation Academy

Gain additional knowledge

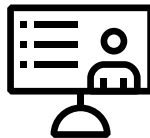
The Automation Academy is responsible for the **targeted training** of our customers as well as our own employees. Expand your skills in the field of automation technology and learn how to independently implement **efficient automation solutions** with B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom training

An experienced trainer will guide you through the learning program. On site at the desired B&R location. Learn individually or in small learning groups.



Virtual classroom

A location-independent distance learning program complements the other learning options we offer. An online tutor accompanies you virtually. The emphasis is on self-study.



Online courses

You acquire your knowledge independently and determine the pace and content yourself. Online courses are available at any time and independent of duration and location.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place!

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

